

Compression d'image par DCT et stockage CSR en C++

Conception du code et représentation des données

Romain Ben Karim Zrig

MAM4 Polytech Nice Sophia | Projet C++

23 septembre 2026

Du projet Python au C++

MAM3 : version Python

- ▶ DCT, quantification, troncature avec `numpy`
- ▶ stockage creux avec `scipy.sparse`
- ▶ application Streamlit

1. Romain

organisation du code
représentation des données

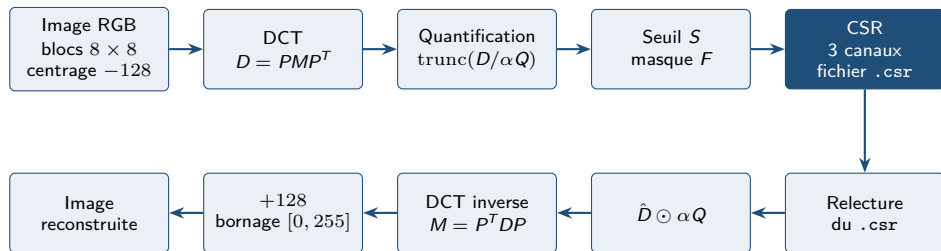
MAM4 : version C++20

- ▶ même algorithme, aucune bibliothèque de calcul
- ▶ but : apprendre le C++ sur un problème connu
- ▶ programme en ligne de commande `jpeg_csr`

2. Karim

matrice creuse, ressources,
polymorphisme, résultats

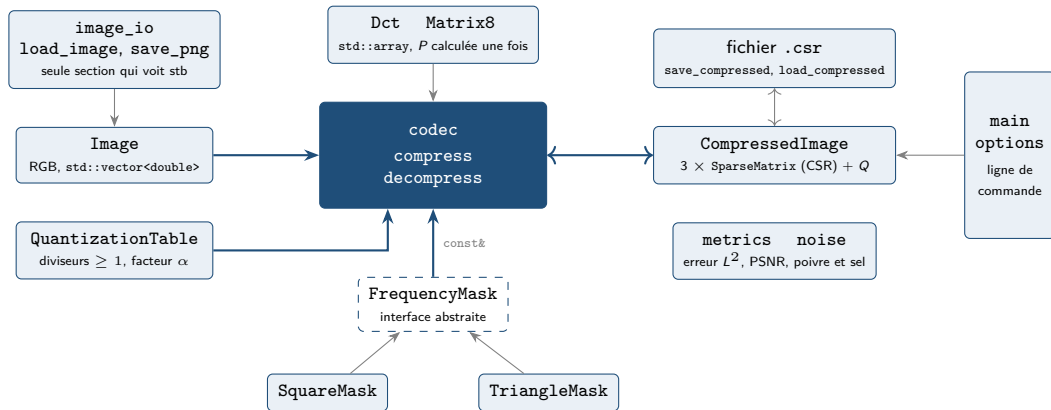
L'algorithme en une slide



P est orthogonale : l'inverse de la DCT est sa transposée.

Masque carré (Python) : $k, l < F$ Masque triangulaire (sujet) : $k + l < F$

Organisation du code

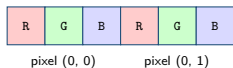


Trois fichiers : `jpeg.hpp` (déclarations commentées), `jpeg.cpp` (définitions, mêmes 12 sections), `main.cpp` | espace de noms `jpeg` | compilation séparée, `-std=c++20 -Wall -Wextra -pedantic`

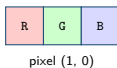
Représenter l'image : un seul `std::vector`

```
class Image {  
public:  
    double& operator()(std::size_t row,  
                        std::size_t col, std::size_t channel);  
    const double& operator()(std::size_t row,  
                             std::size_t col, std::size_t channel) const;  
    // + at(): checked access  
  
private:  
    std::size_t width_;  
    std::size_t height_;  
    std::vector<double> pixels_;  
    // invariant: size == width * height * 3  
};
```

- ▶ pas de vecteur de vecteurs : une seule allocation, lignes toujours de même taille
- ▶ mémoire contiguë, un seul invariant
- ▶ deux surcharges de `operator()` : écrire, ou lire une image `const`
- ▶ `at()` vérifie les bornes, comme `std::vector`



...



...

$$\text{indice} = (\text{row} \times w + \text{col}) \times 3 + \text{channel}$$

Le bloc 8×8 : `std::array`, aucune allocation

```
class Matrix8 {
    ...
private:
    std::array<double, 64> values_{};
};
Matrix8 operator*(const Matrix8& lhs,
                  const Matrix8& rhs);

class Dct {
public:
    Dct();    // computes P and P^T once
    Matrix8 forward(const Matrix8& block) const;
    Matrix8 inverse(
        const Matrix8& coefficients) const;
private:
    // declaration order = initialization order
    Matrix8 p_;
    Matrix8 p_transposed_;
};
```

- ▶ un seul type pour un bloc de pixels, P et les coefficients
- ▶ taille connue à la compilation : pas d'allocation, alors qu'une image 512×512 compte 12 288 blocs
- ▶ `operator*` libre : $p_ * block * p_transposed_$ s'écrit comme la formule
- ▶ piège appris : les membres sont initialisés dans l'ordre de leur **déclaration**

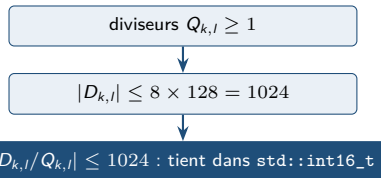
Quantification : un invariant qui fixe un type

```
// Invariant: all divisors are >= 1.
class QuantizationTable {
public:
    // throws std::invalid_argument otherwise
    explicit QuantizationTable(
        const Matrix8& divisors);

    static QuantizationTable standard();
    static QuantizationTable uniform(double value);

    // new table alpha * Q, invariant checked again
    QuantizationTable scaled(double alpha) const;

private:
    Matrix8 divisors_;
};
```



- ▶ vérifié une fois, dans le constructeur
- ▶ explicit : pas de conversion implicite
- ▶ fonctions statiques : une par matrice du projet MAM3

La matrice creuse CSR, écrite à la main

```
class SparseMatrix {
public:
    // dense matrix: only read
    SparseMatrix(std::size_t rows, std::size_t cols,
        const std::vector<std::int16_t>& dense);

    // arrays read in a file: taken by value,
    // then moved (std::move), checked
    SparseMatrix(std::size_t rows, std::size_t cols,
        std::vector<std::int16_t> values,
        std::vector<std::int32_t> column_indices,
        std::vector<std::int32_t> row_pointers);
private:
    std::vector<std::int16_t> values_;
    std::vector<std::int32_t> column_indices_;
    std::vector<std::int32_t> row_pointers_;
};
```

$$\begin{pmatrix} 5 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 3 & 7 \end{pmatrix} \rightarrow \begin{array}{l} \text{values} = \{5, 3, 7\} \\ \text{column_indices} = \{0, 1, 2\} \\ \text{row_pointers} = \{0, 1, 1, 3\} \end{array}$$

- ▶ remplace `scipy.sparse.csr_matrix`, mêmes types
- ▶ environ 95 % de zéros : seuls les autres sont stockés
- ▶ invariant : pointeurs croissants, colonnes triées, aucun zéro stocké
- ▶ déplacement : aucun élément recopié à la relecture

Image compressée et fichier .csr

Composition

- ▶ `CompressedImage` = dimensions + Q + `std::vector` de 3 `SparseMatrix`
- ▶ invariant : multiples de 8, trois canaux de la taille de l'image

Flux binaires

- ▶ `write` et `read` surchargées : le compilateur choisit selon le type
- ▶ `std::ofstream` ferme le fichier tout seul (RAII)

Contenu du fichier	Type
signature JCSR	4 octets
largeur, hauteur	uint32
matrice Q	64 double
pour chaque canal R, G, B :	
nnz	uint32
pointeurs de lignes	int32
colonnes	int32
valeurs	int16

Relecture : dimensions bornées **avant** toute allocation, puis chaque constructeur vérifie son invariant. Un fichier corrompu lance une exception.

Ressources : RAI et règle de zéro

```
// Array allocated by stbi_load: freed by the  
// destructor, even if an exception is thrown.  
class StbPixels {  
public:  
    explicit StbPixels(const std::string& path);  
    ~StbPixels();    // stbi_image_free(data_)  
  
    StbPixels(const StbPixels&) = delete;  
    StbPixels& operator=(const StbPixels&) = delete;  
  
private:  
    int width_{};  
    int height_{};  
    unsigned char* data_{nullptr};  
};
```

Règle de zéro partout ailleurs

- ▶ `std::vector` et `std::array` gèrent la mémoire
- ▶ aucun destructeur ni copie écrits
- ▶ aucun `new` ni `delete` dans le programme

Une seule ressource brute

- ▶ le tableau de pixels de stb, confié à `StbPixels`
- ▶ copie interdite : deux objets libéreraient le même tableau (TD 4)

Polymorphisme : le masque est choisi à l'exécution

```
class FrequencyMask {
public:
    virtual bool keeps(std::size_t k,
                       std::size_t l) const = 0;
    virtual std::string name() const = 0;
    virtual ~FrequencyMask() = default;
};

class SquareMask : public FrequencyMask { ... };
class TriangleMask : public FrequencyMask { ... };

CompressedImage compress(const Image& image,
                        const QuantizationTable& table,
                        int threshold, const FrequencyMask& mask);
```

```
// main.cpp
if (options.mask == MaskKind::triangle) {
    const TriangleMask mask{options.cutoff};
    return compress_with_mask(options, mask);
}
const SquareMask mask{options.cutoff};
return compress_with_mask(options, mask);
```

- ▶ même démarche que ScalarFunction au TD 6
- ▶ compress ne nomme aucun masque : en ajouter un ne la modifie pas
- ▶ objet local passé par référence : pas de new, pas de copie, pas de slicing

Validation et résultats

17

fonctions de test

tests Python repris, invariants, fichier
.csr ; CI sous AddressSanitizer et
UBSan, 91 % de couverture

25

coefficients différents sur 786 432

écart de 1, dû à un arrondi flottant côté
Python ($/255$ puis $\times 255$)

$\times 11$

plus rapide que Python

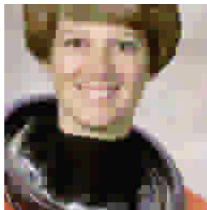
50 ms contre 543 ms pour compresser
et décompresser une image 512×512



originale



$\alpha = 1$: erreur 5,4 %



$\alpha = 5$: erreur 11,3 %



$\alpha = 20$: erreur 33 %

Bilan : ce que le C++ nous a appris

- ▶ **Le type porte l'information** : les hypothèses implicites du Python (multiples de 8, int16, fichier cohérent) deviennent des invariants
- ▶ **Chaque copie se voit** : T, T& ou const T&, std::array ou std::vector, copie ou déplacement
- ▶ **La durée de vie structure le code** : RAI et règle de zéro, aucune gestion mémoire manuelle

Pour aller plus loin

- ▶ zigzag et codage entropique (RLE, Huffman) : un vrai fichier JPEG
- ▶ espace YCbCr, sous-échantillonnage de la chrominance
- ▶ pointeurs intelligents pour conserver plusieurs masques

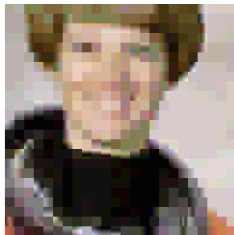
Merci, place aux questions

<https://github.com/BnRomain/jpeg-compression>

Annexe : matrices de quantification



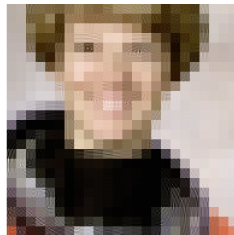
Q standard



Q uniforme (100)



basses fréquences



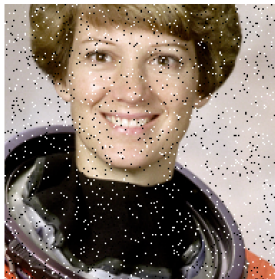
hautes fréquences

	Conservation	Erreur L^2	Taille CSR
Q standard (psychovisuelle)	5,79 %	5,43 %	273 Ko
Q uniforme	1,65 %	13,85 %	82 Ko
diviseurs faibles en basses fréquences	8,76 %	6,52 %	409 Ko
diviseurs faibles en hautes fréquences	25,45 %	17,49 %	1 179 Ko

Annexe : bruit poivre et sel



image pure



bruitée à 5 %



reconstruite, masque triangulaire $F = 4$

Erreur L^2 par rapport à l'image pure :

- ▶ image bruitée : 24,0 %
- ▶ reconstruite, triangle $F = 4$: 11,3 %
- ▶ reconstruite, réglages Python : 12,8 %

Le bruit est fait de variations très rapides, donc de hautes fréquences : la troncature agit comme un filtre passe-bas, au prix d'un flou des contours.

Annexe : utilisation

```
g++ -std=c++20 -O2 -Wall -Wextra -pedantic jpeg.cpp main.cpp -o jpeg_csr
./jpeg_csr compress images/astronaut.png --out results/standard
./jpeg_csr decompress results/standard/astronaut.csr decoded.png
```

```
Image      : images/astronaut.png (512 x 512, cropped to 512 x 512)
Settings   : Q standard, alpha 1.00, threshold 2, mask square F = 6
Coefficients : 45566 non-zero out of 786432 (retention rate 5.79 %)
Quality    : relative L2 error 5.43 %, PSNR 30.49 dB
Dense memory : 6144.00 KiB (float64, like img.nbytes)
CSR memory  : 273.00 KiB (int16 values, int32 indices)
Memory gain : dense / CSR = 22.51x
.csr file   : 273.52 KiB, source image 773.00 KiB
Time        : compression 28.23 ms, decompression 21.55 ms
```

Options : `--table standard|uniform|low|high`, `--alpha`, `--threshold`, `--mask square|triangle`, `--cutoff`, `--noise`, `--out`. Dans le dépôt, `make && make test` construit aussi les 17 tests.